

Глубокое исследование: Архитектура, Протоколы и Реализация MEM_CHECK Spoofing для Warden в WoW 3.3.5a

Введение в защитные механизмы клиента World of Warcraft 3.3.5a

Экосистема World of Warcraft (WoW) версии 3.3.5a (Build 12340), выпущенная в эпоху дополнения *Wrath of the Lich King*, представляет собой классический пример архитектуры «клиент-сервер» конца 2000-х годов, где доверие к клиенту является минимальным, но неизбежным фактором. В условиях, когда игровой клиент исполняется на неконтролируемом оборудовании пользователя, разработчики Blizzard внедрили систему Warden — динамически подгружаемый античит-модуль, предназначенный для верификации целостности памяти процесса, обнаружения стороннего программного обеспечения и предотвращения манипуляций с игровым протоколом.

Для исследователей в области информационной безопасности и разработчиков, занимающихся поддержкой частных серверов (эмуляторов, таких как AzerothCore или TrinityCore), понимание механизмов работы Warden является критически важным. Основной вектор защиты Warden строится на сканировании памяти (Memory Scanning), в частности через механизм MEM_CHECK. Данный механизм позволяет серверу запросить содержимое произвольного участка памяти клиента и сверить его с эталонным значением. Если исследователь использует техники inline-hooking (модификация первых байт функции инструкцией JMP) или software breakpoints (инструкция INT 3 / 0xCC) для анализа трафика или изменения поведения клиента, MEM_CHECK немедленно выявит несоответствие контрольной суммы или сырых байт.

Настоящий отчет посвящен детальному анализу протокола Warden, криптографических примитивов, используемых для защиты коммуникации, и разработке метода обхода (spoofing) проверок типа MEM_CHECK. Предлагаемая методология основывается на концепции «Теневой копии» (Shadow Copy) — техники, при которой ответы на запросы античита формируются не на основе данных из оперативной памяти (которые могут быть модифицированы), а путем чтения оригинальных байт непосредственно из исполняемого файла на диске. Такой подход позволяет сохранить функциональность внедренных модификаций (хуков) при полной невидимости для серверной части античита.

1. Архитектурный анализ системы Warden

Система Warden не является монолитной частью исполняемого файла WoW.exe. Вместо этого она реализована как полиморфный механизм, загружающий исполняемые модули с сервера во время выполнения. Это решение позволяет разработчикам обновлять логику обнаружения читов без необходимости выпускать патчи для всего клиента, а также затрудняет статический анализ, так как модуль античита присутствует в памяти только во время игровой сессии.

1.1. Жизненный цикл сессии Warden

Взаимодействие между клиентом и сервером в контексте Warden можно разделить на несколько ключевых этапов, каждый из которых критичен для успешной реализации спуфинга.

1. **Инициализация (Handshake):** При входе в игровой мир сервер отправляет пакет инициализации, содержащий криптографический "seed" (зерно). Клиент и сервер используют этот seed для генерации сеансовых ключей шифрования.
2. **Загрузка модуля (Module Transfer):** Сервер передает клиенту тело модуля Warden. В версии 3.3.5a это, как правило, бинарный файл, структурно напоминающий DLL, но загружаемый вручную (Manual Map) без использования стандартного Windows API LoadLibrary. Это делается для того, чтобы модуль не отображался в списке загруженных модулей (PEB -> Ldr), что усложняет его обнаружение внешними инструментами.
3. **Активация и Криптография:** Модуль расшифровывается, проверяется его целостность (обычно через MD5 и RSA-подпись), после чего управление передается точке входа модуля.
4. **Цикл проверок (Challenge-Response):** Сервер периодически отправляет пакеты с запросами на проверку (WARDEN_SMSG_CHEAT_CHECKS_REQUEST). Модуль выполняет сканирование и возвращает результаты (WARDEN_CMSG_CHEAT_CHECKS_RESULT).

1.2. Классификация проверок

Анализ исходного кода эмуляторов серверов (AzerothCore, TrinityCore) ¹ позволяет выделить основные типы проверок, используемых в протоколе:

Тип (Hex)	Идентификатор	Описание	Размер запроса
0xF3	MEM_CHECK	Проверка байт по указанному адресу памяти.	1 + 4 + 1 байт
0xB2	PAGE_CHECK_A	Хеширование (SHA-1) целой	4 + 20 + 4 + 1 байт

		страницы памяти (4KB).	
0xBF	PAGE_CHECK_B	Аналог типа А, но сканирует только страницы с заголовками MZ+PE.	4 + 20 + 4 + 1 байт
0x98	MPQ_CHECK	Проверка целостности файлов игровых архивов (.MPQ).	1 байт (индекс)
0xD9	MODULE_CHECK	Поиск внедренных DLL сторонних разработчиков.	4 + 20 байт
0x8B	LUA_EVAL_CHECK	Выполнение произвольного Lua-кода и возврат результата.	Переменный

Для задач скрытия хуков (inline hooks) наибольшую угрозу представляет именно MEM_CHECK (0xF3) и, в меньшей степени, PAGE_CHECK. MEM_CHECK позволяет серверу точно проверить пролог функции (например, Direct3D9::EndScene или NetClient::SendPacket), где обычно располагаются инструкции безусловного перехода (JMP), установленные читом.

2. Криптографический слой и транспортный протокол

Любая попытка манипуляции данными Warden требует преодоления криптографического слоя. В версии 3.3.5a используется потоковый шифр RC4 (Rivest Cipher 4), который применяется ко всему содержимому пакетов Warden (как входящих, так и исходящих), за исключением заголовка опкода самого пакета WoW.

2.1. Алгоритм генерации ключей (SHA1 XOR-Fold)

Ключ RC4 не передается в открытом виде. Он генерируется на основе обмена случайными числами (nonce/seed). Процесс генерации ключа, подтвержденный анализом исходного кода ядра AzerothCore¹, включает использование алгоритма SHA-1 с последующей операцией свертки (XOR-folding).

Ключ сессии $K_{session}$ вычисляется следующим образом:

1. **Формирование входного буфера:** Конкатенация ServerSeed (16 байт) и ClientSeed (16 байт), а также, в некоторых случаях, ModuleKey (ключа самого модуля).
2. **Хеширование:** Вычисляется $H = \text{SHA1}(\text{Buffer})$. Результат SHA-1 имеет длину 20 байт (160 бит).
3. **Свертка (XOR-Fold):** Поскольку RC4 в данной реализации часто инициализируется 16-байтным ключом, 20 байт хеша преобразуются в 16 байт. Обычно это делается путем отсечения последних 4 байт или их наложением (XOR) на начало массива, однако в реализации Warden 3.3.5a часто используется просто первые 16 байт хеша или специфическая перестановка.

Важнейшим аспектом реализации является тот факт, что RC4 — это потоковый шифр. Это означает, что состояние шифра (S-box и индексы i, j) изменяется с каждым зашифрованным или расшифрованным байтом.³

Критическая уязвимость реализации: Если мы попытаемся расшифровать пакет Warden для анализа содержимого, мы изменим внутреннее состояние RC4. Когда настоящий модуль Warden (который мы пытаемся обмануть) попытается расшифровать этот же пакет, его состояние будет рассинхронизировано с состоянием, ожидаемым сервером, или с данными в буфере. Результатом станет некорректная расшифровка всех последующих пакетов, что приведет к разрыву соединения или бану за отправку "мусорных" данных.

Следовательно, для реализации подмены (spoofing) необходимо поддерживать **параллельный контекст RC4** (Shadow Context). Мы должны перехватывать зашифрованные данные, расшифровывать их в *нашем* контексте для анализа, а затем, если мы решаем подменить ответ, мы должны сгенерировать ответ, зашифровать его *нашим* контекстом и отправить, полностью исключив оригинальный модуль Warden из цепочки обработки данного пакета. Либо же мы должны внедряться до момента шифрования/дешифрования, что сложнее из-за ручной загрузки модуля.

2.2. Структура пакетов SMSG_WARDEN_DATA

Пакеты Warden инкапсулируются внутри стандартных пакетов протокола WoW.

- **SMSG_WARDEN_DATA (Opcode 0x2E6):** Данные от сервера к клиенту.
- **CMSG_WARDEN_DATA (Opcode 0x2E7):** Данные от клиента к серверу.

Полезная нагрузка пакета SMSG_WARDEN_DATA после расшифровки RC4 имеет следующую структуру:

1. **Warden Opcode (1 байт):** Определяет подтип операции. Для запроса проверок это 0x02 (WARDEN_SMSG_CHEAT_CHECKS_REQUEST).⁵
2. **Данные (Variable):** Последовательность байт, зависящая от опкода.

В случае WARDEN_SMSG_CHEAT_CHECKS_REQUEST, данные представляют собой поток проверок, идущих друг за другом. В отличие от многих других протоколов, здесь часто отсутствует поле "количество проверок". Парсинг осуществляется последовательно: читается длина проверки, затем тело проверки, затем длина следующей, пока пакет не закончится.

3. Анатомия проверки MEM_CHECK (0xF3)

Для успешной подмены необходимо досконально понимать структуру запроса MEM_CHECK. Основываясь на данных из репозитория эмуляторов ¹, структура одной записи проверки в потоке выглядит следующим образом:

Смещение	Тип	Значение / Имя	Описание
0x00	uint8	Length	Общая длина данных проверки (не включая этот байт). Для MEM_CHECK обычно 7 байт.
0x01	uint8	Type	0xF3 (MEM_CHECK)
0x02	uint8	ModuleIndex	Индекс модуля. 0 = основной .exe, другие значения — загруженные DLL.
0x03	uint32	Offset	Смещение относительно базового адреса модуля

			(ModuleBase + Offset).
0x07	uint8	ByteCount	Количество байт, которые необходимо прочитать и отправить (Len).

Алгоритм работы оригинального модуля:

1. Получает ModuleIndex. Если 0, берет GetModuleHandle(NULL) (адрес загрузки WoW.exe).
2. Вычисляет адрес чтения: Target = BaseAddress + Offset.
3. Читает ByteCount байт по адресу Target.
4. Добавляет прочитанные байты в выходной буфер результатов.

Алгоритм валидации на сервере: Сервер хранит в базе данных таблицу warden_checks.⁶ В ней для каждого ID проверки хранится ожидаемый результат (result) или данные для сравнения (data). При получении ответа от клиента сервер сравнивает присланные байты с эталонными.

4. Методология «Теневой Копии» (Shadow Copy)

Суть метода заключается в перенаправлении операций чтения памяти, инициированных Warden, на "чистую" копию исполняемого файла, загруженную с диска. Поскольку мы модифицировали секцию .text в памяти процесса (вставили хуки), чтение из оперативной памяти выдаст измененные байты (0xE9... или 0xCC). Чтение же из файла на диске вернет оригинальные машинные инструкции, которые ожидает увидеть сервер.

Однако прямая подмена адреса чтения невозможна, так как модуль Warden находится внутри процесса и имеет полный доступ к адресному пространству. Мы не можем "запретить" ему читать память. Поэтому мы должны перехватить сам запрос (пакет), распарсить его, самостоятельно прочитать нужные данные из файла и сформировать ответный пакет, имитируя работу модуля.

4.1. Проблема трансляции адресов (RVA vs File Offset)

Ключевая техническая сложность заключается в несоответствии адресов в памяти и смещений в файле. Исполняемые файлы формата PE (Portable Executable) имеют

различную структуру выравнивания на диске и в памяти.

- **Virtual Address (VA):** Абсолютный адрес в памяти (например, 0x00401000).
- **Relative Virtual Address (RVA):** Смещение относительно базы загрузки (VA - ImageBase). Warden обычно оперирует RVA (передавая Offset).
- **Raw File Offset:** Смещение байта от начала файла на диске.

В памяти секции выравниваются по границе SectionAlignment (обычно 0x1000 или 4096 байт). На диске они выравниваются по FileAlignment (обычно 0x200 или 512 байт). Следовательно, мы не можем просто прочитать файл по смещению Offset. Нам необходимо реализовать полноценный PE-парсер.

4.2. Алгоритм трансляции адресов

Для корректного чтения байта, соответствующего RVA X, из файла на диске, необходимо выполнить следующие шаги:

1. **Парсинг заголовков:** Считать IMAGE_DOS_HEADER, перейти к IMAGE_NT_HEADERS.
2. **Таблица секций:** Получить доступ к массиву IMAGE_SECTION_HEADER.
3. **Поиск секции:** Итерировать по секциям, проверяя условие:

$$\text{Section.VirtualAddress} \leq \text{RVA} < \text{Section.VirtualAddress} + \text{Section.Misc.VirtualSize}$$

4. **Вычисление смещения:** Если RVA попадает в секцию, вычислить дельту:

$$\Delta = \text{RVA} - \text{Section.VirtualAddress}$$

5. **Получение файлового смещения:**

$$\text{FileOffset} = \text{Section.PointerToRawData} + \Delta$$

6. **Чтение:** Считать данные из файла по позиции FileOffset.

4.3. Обработка релокаций (Relocations)

В версии WoW 3.3.5a (Build 12340) ASLR (Address Space Layout Randomization) обычно отключен или не используется эффективно старыми компиляторами. Базовый адрес WoW.exe фиксирован (0x00400000). Это упрощает задачу, так как Offset в пакете Warden, скорее всего, является жестким смещением от этой базы. Если бы использовался ASLR, нам пришлось бы учитывать дельту ребазинга, но в контексте 3.3.5a можно считать, что Offset = RVA.

Тем не менее, важно проверить флаг IMAGE_DLLCHARACTERISTICS_DYNAMIC_BASE в OptionalHeader. Если он установлен, то реальный адрес в памяти может отличаться, но Warden запрашивает смещение *относительно модуля*, поэтому наша логика работы с файлом (который является "образом" модуля) остается верной, так как RVA инвариантен к базе загрузки.

5. Практическая реализация Spoofing-механизма

Ниже представлен детальный план реализации на C++ для внедряемой DLL.

5.1. Инициализация и загрузка чистого образа

При загрузке нашей DLL (DllMain), мы должны открыть оригинальный файл WoW.exe и загрузить его в память для быстрого доступа. Чтение с диска при каждом пакете может создать задержки, которые детектируются через TIMING_CHECK.

C++

```
#include <windows.h>
#include <vector>
#include <fstream>
#include <iostream>

// Структура для хранения данных о секциях
struct SectionInfo {
    uint32_t virtualAddr;
    uint32_t virtualSize;
    uint32_t rawOffset;
};

class ShadowImage {
private:
    std::vector<uint8_t> fileBuffer;
    std::vector<SectionInfo> sections;
    PIMAGE_NT_HEADERS ntHeaders;

public:
    bool Initialize(const std::wstring& path) {
        // Чтение файла в буфер
        std::ifstream file(path, std::ios::binary | std::ios::ate);
        if (!file.is_open()) return false;
        size_t fileSize = file.tellg();
        file.seekg(0, std::ios::beg);
        fileBuffer.resize(fileSize);
        file.read((char*)fileBuffer.data(), fileSize);
    }
};
```

```

// Парсинг PE заголовков
PIMAGE_DOS_HEADER dosHeader = (PIMAGE_DOS_HEADER)fileBuffer.data();
if (dosHeader->e_magic!= IMAGE_DOS_SIGNATURE) return false;

ntHeaders = (PIMAGE_NT_HEADERS)(fileBuffer.data() + dosHeader->e_lfanew);
if (ntHeaders->Signature!= IMAGE_NT_SIGNATURE) return false;

// Сохранение информации о секциях
PIMAGE_SECTION_HEADER secHeader = IMAGE_FIRST_SECTION(ntHeaders);
for (int i = 0; i < ntHeaders->FileHeader.NumberOfSections; i++) {
    SectionInfo info;
    info.virtualAddr = secHeader[i].VirtualAddress;
    info.virtualSize = secHeader[i].Misc.VirtualSize;
    info.rawOffset = secHeader[i].PointerToRawData;
    sections.push_back(info);
}
return true;
}

bool ReadBytes(uint32_t rva, size_t length, std::vector<uint8_t>& out) {
    // Поиск секции
    for (const auto& sec : sections) {
        if (rva >= sec.virtualAddr && rva < sec.virtualAddr + sec.virtualSize) {
            // Проверка границ
            if (rva + length > sec.virtualAddr + sec.virtualSize) return false;

            uint32_t offsetInSection = rva - sec.virtualAddr;
            uint32_t fileOffset = sec.rawOffset + offsetInSection;

            if (fileOffset + length > fileBuffer.size()) return false;

            out.resize(length);
            memcpy(out.data(), &fileBuffer[fileOffset], length);
            return true;
        }
    }

    // Если RVA не в секции (например, в заголовке), можно читать напрямую,
    // если RVA < SizeOfHeaders.
    return false;
}
};

```

5.2. Перехват трафика Warden

Необходимо установить хук на функцию обработки входящих пакетов. В 3.3.5а это часто NetClient::ProcessMessage или более специфичная функция диспетчеризации опкодов.

Когда приходит пакет с Opcode == MSG_WARDEN_DATA (0x2E6):

1. **Расшифровка (Shadow Context):** Используем нашу копию ключа RC4 для расшифровки тела пакета.
2. **Анализ:** Проверяем первый байт расшифрованных данных. Если это 0x02 (CHEAT_CHECKS_REQUEST), начинаем процесс подмены.
3. **Обработка потока проверок:**
Итерируем по буферу. Читаем длину проверки. Читаем данные.
 - Если тип 0xF3 (MEM_CHECK):
 - Извлекаем Offset и Length.
 - Вызываем ShadowImage::ReadBytes(Offset, Length).
 - Полученные "чистые" байты записываем в буфер ответа.
 - Если тип 0xB2/0xBF (PAGE_CHECK):
 - Извлекаем адрес страницы.
 - Читаем 4096 байт из ShadowImage.
 - Считаем SHA-1 хеш от этих байт.
 - Записываем хеш в буфер ответа.
 - Если тип другой (MPQ, LUA):
 - Необходимо реализовать эмуляцию или проксировать выполнение, но с осторожностью. Для LUA проверок часто безопаснее вернуть пустую строку или ошибку, если они не критичны, либо выполнить их, убедившись, что Lua-движок не пропатчен.
4. **Формирование ответа:**
Собираем результаты всех проверок в пакет MSG_WARDEN_DATA.
Шифруем его *нашим* контекстом RC4.
Отправляем серверу.
5. **Блокировка:** Возвращаем true (или прерываем выполнение) в хуке, чтобы *оригинальный* код клиента не получил этот пакет. Это предотвратит двойной ответ и рассинхронизацию "настоящего" RC4 (который останется в старом состоянии, но сервер будет получать данные, зашифрованные нашим синхронизированным контекстом).

5.3. Проблема RC4 Key Initialization

Где взять ключ для нашего Shadow Context? Самый надежный способ — перехватить момент генерации ключа. Функция Warden::InitializeModule или её аналог отвечает за хеширование seed'a.

Мы можем установить хук на криптографическую функцию (например, на внутреннюю

реализацию SHA1_Final или функцию установки ключа RC4 SARC4_SetKey).

C++

```
// Псевдокод хука установки ключа
void __stdcall Hooked_SARC4_SetKey(uint8_t* key, int len) {
    // Сохраняем ключ в наш контекст
    MyShadowRC4.Init(key, len);

    // Вызываем оригинал, чтобы клиент продолжил работу (если мы не планируем полностью убить
    // модуль Warden)
    // Но если мы полностью эмулируем Warden, оригинал вызывать не обязательно,
    // однако это может сломать логику клиента, если он использует RC4 для чего-то еще.
    Original_SARC4_SetKey(key, len);
}
```

В контексте полной эмуляции (когда мы полностью подменяем модуль), мы просто реализуем handshake сами, генерируем ключ и начинаем общение, игнорируя загрузку бинарника с сервера. Но в задаче "Defensive Research" чаще используется подход "Man-in-the-Middle" внутри процесса: мы даем модулю загрузиться, но фильтруем его входящие пакеты.

6. Серверная валидация и риски обнаружения

Понимание того, как сервер проверяет наши ответы, позволяет избежать глупых ошибок. В исходном коде TrinityCore/AzerothCore проверка результата MEM_CHECK выглядит тривиально:

C++

```
// Пример логики сервера (AzerothCore)
if (check->Type == MEM_CHECK) {
    if (memcmp(receivedData, expectedData, check->Length) != 0) {
        // Ошибка проверки!
        Penalty(player);
    }
}
```

```
}  
}
```

Сервер ожидает *точное* совпадение байт. Это означает, что версия нашего файла WoW.exe на диске должна *бит-в-бит* соответствовать версии, которую ожидает сервер (Build 12340). Любое отличие в патче или локализации может привести к бану.

6.1. Тайминг-атаки (Timing Checks)

Если сервер использует TIMING_CHECK (хотя в стандартном 3.3.5a они редки или отключены), он замеряет время между отправкой запроса и получением ответа.

- Чтение из оперативной памяти (L1/L2 кэш) занимает наносекунды.
- Чтение с HDD/SSD может занимать миллисекунды.
- **Решение:** Полная предзагрузка WoW.exe в оперативную память при старте чита (как показано в коде ShadowImage::Initialize). Чтение из std::vector в памяти будет неотличимо по скорости от чтения из реальной памяти процесса.

6.2. Само-проверка (Self-Check)

Сервер может запросить проверку памяти не модуля WoW.exe, а *самого модуля Warden* или памяти, где располагается наш хук (если он вне .text).

- Если запрос идет с ModuleIndex!= 0, нужно определить, какой это модуль.
- Если это системная DLL (kernel32.dll), мы можем загрузить её с диска.
- Если это сам модуль Warden (который скачан с сервера), у нас должна быть его копия. Так как модуль динамический, мы должны перехватывать его скачивание (MSG_WARDEN_DATA с опкодом 0x00 - Module Info/Load), сохранять его тело и использовать для ответов на проверки его собственной целостности.

6.3. Контекстные проверки

Некоторые проверки могут быть контекстно-зависимыми (например, проверить переменную, которая меняется в ходе игры, а не статический код). MEM_CHECK обычно используется для кода (статического), но если сервер запросит адрес переменной (секция .data), наш метод вернет начальное значение из файла, а не текущее игровое.

- **Риск:** Если сервер проверяет динамические переменные через MEM_CHECK (например, "флаг полета"), наш метод вернет "земля", хотя игрок летит. Это вызовет несоответствие, если сервер ожидает текущее значение.
- **Анализ:** В 3.3.5a MEM_CHECK используется преимущественно для проверки целостности кода (.text) и таблиц импорта/экспорта (.rdata). Проверка игровых состояний обычно идет через анализ пакетов движения, а не чтение памяти. Тем не менее, для полной безопасности следует добавить фильтр: если Section является Writeable (на запись, например .data, .bss), читать из *реальной* памяти, а не из Shadow Copy. Если секция Executable (.text), читать из Shadow Copy.

7. Расширенные данные и сравнительный анализ

Для обеспечения полноты исследования, ниже приведена сводная таблица критических структур данных, которые необходимо учитывать при реализации.

7.1. Структура базы данных проверок (Server-Side)

Таблица `warden_checks` в базе данных сервера (MySQL) является источником истины для античита.

Поле	Тип данных	Назначение
id	smallint	Уникальный идентификатор проверки.
type	tinyint	Тип проверки (см. раздел 1.2).
input (data)	blob	Данные запроса (смещение, длина).
result	blob	Ожидаемые данные (эталонные байты или хеш).
comment	varchar	Описание (например, "Check for Hitchhiker's Hack").

Понимание этой структуры подтверждает, что валидация происходит путем простого бинарного сравнения (`memcmp`) на стороне сервера. Никакой сложной эвристики на стороне сервера для `MEM_CHECK` в стандартных ядрах 3.3.5a не предусмотрено.⁶

7.2. Сравнение методов обхода

Метод	Сложность	Надежность	Примечание
Отключение хука	Высокая	Низкая	Требует идеальной

перед проверкой			синхронизации потоков. Риск race condition (хук снят -> пакет игры прошел -> краш или детект).
Shadow Copy (Теневая копия)	Средняя	Высокая	Полная изоляция чита от античита. Не требует снятия хуков. Работает для всех статических проверок.
Эмуляция Warden (Полная)	Очень высокая	Максимальная	Требует полного реверс-инжиниринга модуля. Сложно поддерживать при обновлении модуля сервером.

8. Заключение

Реализация подмены ответов MEM_CHECK для Warden в клиенте WoW 3.3.5a является классической задачей в области защитных исследований игровых протоколов. Представленный в отчете метод «Теневой копии» (Shadow Copy) демонстрирует высокую эффективность против статических проверок целостности кода.

Ключевыми факторами успеха реализации являются:

1. **Прозрачность:** Использование Shadow Context для RC4 шифрования позволяет незаметно для сервера подменять содержимое пакетов.
2. **Точность:** Корректная трансляция адресов RVA в файловые смещения через парсинг PE-структуры гарантирует, что возвращаемые байты будут идентичны тем, что сервер ожидает увидеть от "чистого" клиента.
3. **Производительность:** Предварительная загрузка образа файла в память нейтрализует векторы атак, основанные на измерении времени отклика (TIMING_CHECK).

Данное исследование подтверждает, что архитектура Warden версии 3.3.5a, полагающаяся на проверку целостности клиента на стороне клиента (Client-side trust), фундаментально уязвима для атак типа Man-in-the-Middle внутри процесса, при условии корректной изоляции среды исполнения античита от модифицированной среды игры.

Список использованных технических источников

- ¹ — Реализация WardenWin и структура пакетов в AzerothCore.
- ² — Определения опкодов Warden (0xF3, 0xB2 и др.).
- ⁵ — Парсинг пакетов MSG_WARDEN_DATA и инициализация криптографии.
- ³ — Описание алгоритма потокового шифрования RC4 и уязвимостей состояния.
- ⁶ — Структура базы данных warden_checks и серверная логика валидации.
- ⁹ — Формирование пакетов запросов на стороне сервера.
- ¹⁰ — Механизмы валидации модулей Warden (MD5/RSA).

Конец отчета.

Works cited

1. AzerothCore WardenWin Implementation | PDF | Gnu | Software Development - Scribd, accessed February 15, 2026, <https://www.scribd.com/document/582975372/WardenWin>
2. Warden.h File Reference - AzerothCore Doxygen, accessed February 15, 2026, https://www.azerothcore.org/doxygen/d7/dc7/Warden_8h.html
3. RC4 Encryption Algorithm Stream Ciphers Defined - Okta, accessed February 15, 2026, <https://www.okta.com/identity-101/rc4-stream-cipher/>
4. RC4 - Wikipedia, accessed February 15, 2026, <https://en.wikipedia.org/wiki/RC4>
5. WoWTools/src/WoWPacketViewer/Parsers/Warden ... - GitHub, accessed February 15, 2026, https://github.com/tomrus88/WoWTools/blob/master/src/WoWPacketViewer/Parsers/Warden/MSG_WARDEN_DATA.cs
6. [3.3.5]-AccLeiTo(AccLua) Fix script(Warden) #26053 - GitHub, accessed February 15, 2026, <https://github.com/TrinityCore/TrinityCore/issues/26053>
7. Índice del espacio - TrinityCore - Confluence, accessed February 15, 2026, <https://trinitycore.atlassian.net/wiki/spaces/tc/pages/2884304920/page+index>
8. mangos_warden/src/game/WardenMac.cpp at master - GitHub, accessed February 15, 2026, https://github.com/tomrus88/mangos_warden/blob/master/src/game/WardenMac.cpp
9. Kick/Ban when Warden-Checks get interrupted · Issue #17453 - GitHub, accessed February 15, 2026, <https://github.com/azerothcore/azerothcore-wotlk/issues/17453>
10. Wiki: Warden Modules | SkullSecurity Blog, accessed February 15, 2026,

https://www.skullsecurity.org/wiki/Warden_Modules